

The SPO tree: the core algorithm, block by block

Growth and pruning as pseudo-code, as `spo_tree.py` runs them

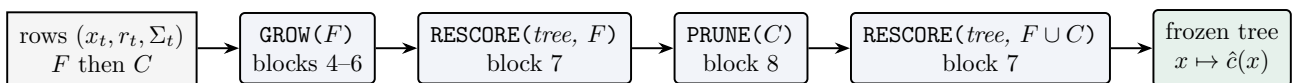
25 September 2026

0. Notation and state

symbol	meaning
<code>t = 1..T</code>	decision dates, in time order ; F = the first 75% (fitting rows), C = the last 25% (checking rows)
<code>x_t, r_t, Σ_t</code>	features on date t ; h -day simple returns of the n assets (cash = 0) ; h -day covariance from the past only
<code>w*(c; u, Σ)</code>	the optimizer: $\arg \max_{w \in \mathcal{W}} c^\top w - f^\top w - u - \lambda w^\top \Sigma w$, $\mathcal{W} = \{0 \leq w_i \leq \bar{w}, \mathbf{1}^\top w = 1\}$, solved exactly because concave
<code>utility(w; r, u, Σ)</code>	$r^\top w - f^\top w - u - \lambda w^\top \Sigma w$
<code>U[t], B[t]</code>	training state, one row per date: holdings before decision t ; hindsight utility from those holdings. Set by MEASURE , frozen between two calls
<code>leaf ℓ = (node, rows, ω, depth)</code>	node: score $\hat{c}_\ell \in [-b, b]^n$, mass n_ℓ , regret sum L_ℓ ; rows: the dates reaching ℓ ; ω : their weights
<code>split node</code>	feature j , threshold θ , two children ; a soft node also holds thresholds $\theta_1 < \dots < \theta_K$ and weights π_k
<code>path (of a tree, over rows)</code>	the trajectory the tree would have traded over those rows, produced by REPLAY : per date, the holdings before the decision, the decision w_t , the fee, the net return, the row regret; plus the Sharpe ratio S of the net returns. Not a root-to-leaf route in the tree
<code>S</code>	the Sharpe ratio of the current tree's path over the fitting rows (in-sample)

Parameters of v1: $\lambda = 1$, $f = (3 \text{ bp}, 0)$, $\bar{w} = 1$, $b = 10\%$, m = minimum number of rows in a leaf = 20, φ = minimum fraction of rows from the parent in a leaf = 0.10, D = maximum depth = 3, shortlist splits (thresholds) 3, ε_R = minimum regret loss improvement = 10^{-6} , ε_S = minimum sharpe improvement = 0, k_p = pruning hurdle = 2, D_p = pruning depth = 1, deciles + refinement with $k = 1$ (refinement hurdle: the refined threshold replaces the best decile only if its regret is lower by more than k standard errors of the difference), κ = soft-split temperature = 0.

1 MAIN



Algorithm 1: MAIN: SPOPortfolioTree.fit

Input: rows $1..T$ in time order; F = the first 75% (fitting), C = the last 25% (checking)

```
1 tree ← GROW( $F$ ) ; // splits and leaf scores, from  $F$  only
2 RESCORE( $tree, F$ ) ; // leaf scores recomputed on the final path; kept only if the
  Sharpe on  $F$  does not drop
3 PRUNE( $tree, C$ ) ; // remove every split that  $C$  does not confirm
4 RESCORE( $tree, F \cup C$ ) ; // leaf scores recomputed on all rows; always kept
Output: tree: split rules  $(j, \theta)$  and one score  $\hat{c}_\ell$  per leaf
```

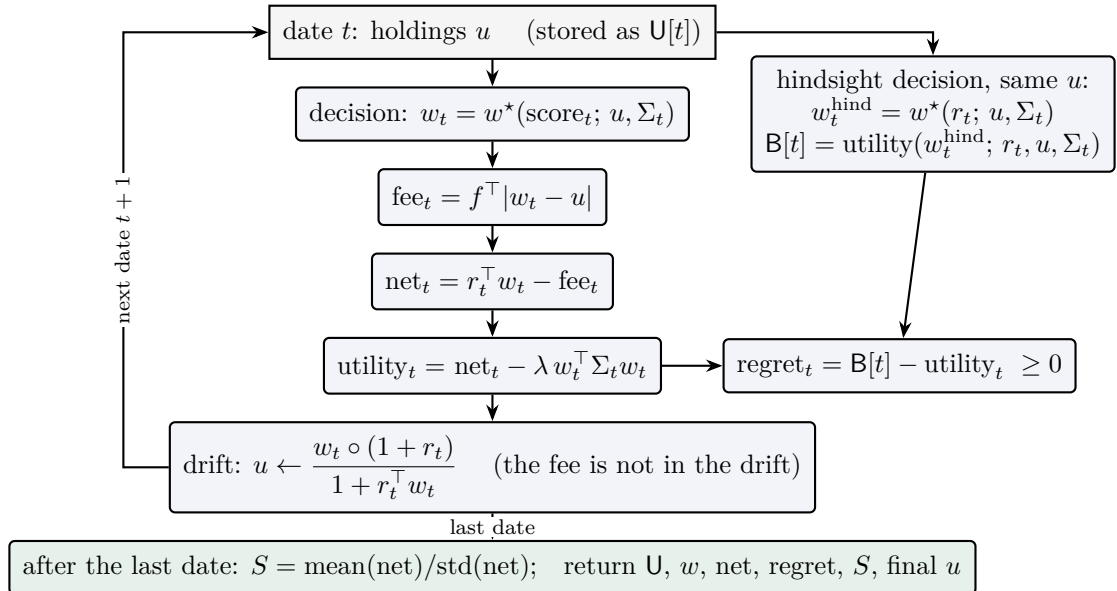
2 REPLAY: the path, the fees, the row regrets

Path. Feeding a tree with the rows $t = 1, 2, \dots$ in time order, each date's decision starting from the holdings the previous date left, gives the trajectory the tree would have traded: holdings, decision, fee, net return and regret on every date, and the Sharpe ratio of the net returns. This trajectory is the *path* of the tree over those rows.

Algorithm 2: ROUTE($tree, rows$) and SCORES($tree, rows$): the tree as one score per date

```
1 function ROUTE( $tree, rows$ ):
2   foreach date  $t$  in rows do
3     send  $x_t$  down the tree: at a split node " $x_j \leq \theta$ ?" go left if  $x_{tj} \leq \theta$ , else right, until a
      leaf  $\ell(t)$  is reached;
4   return  $rows_\ell = \{t : \ell(t) = \ell\}$  for every leaf  $\ell$  ; // the dates each leaf receives
5 function SCORES( $tree, rows$ ):
6   ROUTE( $tree, rows$ );  $scores_t \leftarrow \hat{c}_{\ell(t)}$  for every date  $t$  in rows;
7   return the matrix  $(scores_t)_{t \in rows}$ , one row per date ; // soft splits:
       $scores_t = \sum_\ell \omega_{t\ell} \hat{c}_\ell$ 
```

Fees are computed in REPLAY and nowhere else. One date:



Algorithm 3: REPLAY(scores, rows, u_1) (replay_path) and MEASURE

```
1  $u \leftarrow u_1$ ;
2 for  $t$  in rows, in time order do
3    $U[t] \leftarrow u$ ;  $w_t \leftarrow w^*(\text{scores}_t; u, \Sigma_t)$ ;
4    $\text{fee}_t \leftarrow f^\top |w_t - u|$ ;  $\text{net}_t \leftarrow r_t^\top w_t - \text{fee}_t$ ;
5    $u \leftarrow w_t \circ (1 + r_t) / (1 + r_t^\top w_t)$ ;
6  $\text{utility}_t \leftarrow r_t^\top w_t - \text{fee}_t - \lambda w_t^\top \Sigma_t w_t$ ;  $B[t] \leftarrow \text{utility}(w^*(r_t; U[t], \Sigma_t); r_t, U[t], \Sigma_t)$ ;
    $\text{regret}_t \leftarrow B[t] - \text{utility}_t$ ;
7  $S \leftarrow \text{mean}(\text{net}) / \text{std}(\text{net})$ ; // 0 if net  $\equiv$  0
8 return path = ( $U, w, \text{net}, \text{regret}, S$ , final  $u$ );
9 function MEASURE(path):
10    $U \leftarrow \text{path}.U$ ;  $B \leftarrow \text{path}.utility + \text{path}.regret$ ;  $S \leftarrow \text{path}.S$ ; // holdings frozen from
   here
```

3 The row regret and the score search of a leaf

Algorithm 4: REG(c, t) (_regret_rows) and FIT_LEAF(rows, ω, c_{alt}) (_fit_leaf)

```
1 function REG( $c, t$ ):
2    $w \leftarrow w^*(c; U[t], \Sigma_t)$ ;
3   return  $\max(B[t] - \text{utility}(w; r_t, U[t], \Sigma_t), 10^{-6})$ ;
4  $L(c) := \sum_{t \in \text{rows}} \omega_t \text{REG}(c, t)$ ; // leaf regret of a score  $c$ : one regret per date,
   summed
5 function FIT_LEAF(rows,  $\omega, c_{\text{alt}}$ ):
   // goal: the score  $c \in [-b, b]$  with the smallest  $L(c)$ ; a grid search, refined
   twice
6    $c \leftarrow \text{clip}(\sum_{t \in \text{rows}} \omega_t r_t / \sum_{t \in \text{rows}} \omega_t, [-b, b])$ ; // start 1: the mean return
7   if  $c_{\text{alt}}$  is given and  $L(c_{\text{alt}}) < L(c)$  then
8      $c \leftarrow c_{\text{alt}}$ ; // start 2: an alternative score, see below
9   for  $h$  in ( $b, b/3, b/18$ ) do
10    grid  $\leftarrow$  7 equally spaced values in  $[c - h, c + h] \cap [-b, b]$ ;
11     $c \leftarrow$  the grid value with the smallest  $L$ , if smaller than  $L(c)$ ; // otherwise  $c$  stays
12   return ( $c, L(c)$ );
```

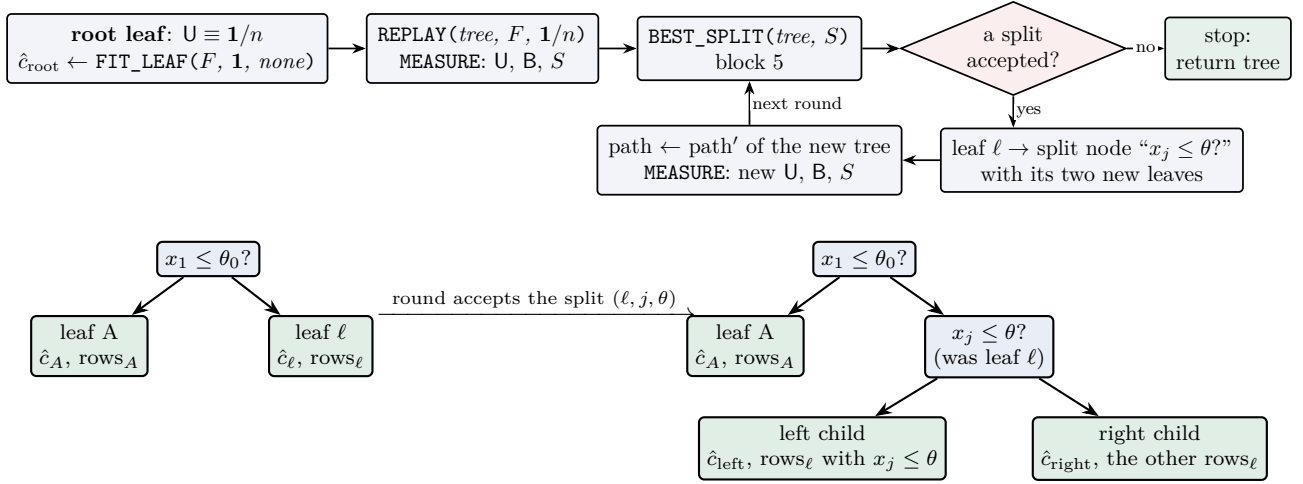
The three passes, with $b = 10\%$: pass 1 covers $[-10\%, 10\%]$ in steps of 3.3%; pass 2 covers $\pm 3.3\%$ around the best value in steps of 1.1%; pass 3 covers $\pm 0.56\%$ in steps of 0.19%. So a leaf score is a multiple of 0.19% near the best point. Cost: $2 + 3 \cdot 7$ evaluations of L per coordinate. The code runs the passes on the asset score and on the cash score in turn; only their difference matters, so everything here is written for the asset score with cash at 0.

The alternative start c_{alt} . A second candidate offered to the search, kept only if it beats the mean-return start:

- root: none;
- child of a candidate split: the score of the leaf being split (its yardstick $P.\hat{c}$), so that a child may simply inherit its parent's score.
- a leaf whose score is searched again: its current score, so that the new search cannot end worse than the score it replaces.

4 GROW

Growth adds one split per round. A round has three parts: every leaf proposes its best candidate splits with their regret reduction Δ (BEST_SPLIT, block 5); the candidates of all leaves are ranked by Δ and tried in that order on the whole tree, the first one whose path raises S being accepted; the path of the accepted tree becomes the reference for the next round (its holdings and hindsight utilities go into U, B). Growth stops when no candidate passes.



Algorithm 5: GROW(F) ($_grow$)

Input: the fitting rows F ; the training-state tables U, B (one row per date) and the Sharpe S are written here

// the root: one leaf holding all fitting rows

1 $U[t] \leftarrow 1/n$ for every $t \in F$; // no path exists yet: every date starts from equal weights

2 $B[t] \leftarrow \text{utility}(w^*(r_t; 1/n, \Sigma_t); r_t, 1/n, \Sigma_t)$ for every $t \in F$; // hindsight utility from equal weights

3 $\hat{c}_{\text{root}} \leftarrow \text{FIT_LEAF}(F, \omega \equiv 1, \text{none})$; // one score for all fitting rows (Algorithm 3)

4 $\text{tree} \leftarrow$ a single leaf: the root, with score $\hat{c}_{\text{root}}$ and rows F ;

5 $\text{path} \leftarrow \text{REPLAY}(\text{SCORES}(\text{tree}, F), F, u_1 = 1/n)$; // trade the root score date by date (Algorithm 2)

6 $\text{MEASURE}(\text{path})$; // $U, B \leftarrow$ the path's holdings and hindsight utilities; $S \leftarrow \text{path}.S$

// the rounds: one accepted split per round

7 **while true do**

8 $\text{result} \leftarrow \text{BEST_SPLIT}(\text{tree}, S)$; // Algorithm 5: the best admissible split of any leaf, with the path of the tree that contains it, or none

9 **if result = none then**

10 **break**; // no candidate passed both gates

11 **else**

12 $(\ell, j, \theta, \text{left child}, \text{right child}, \text{path}') \leftarrow \text{result}$;

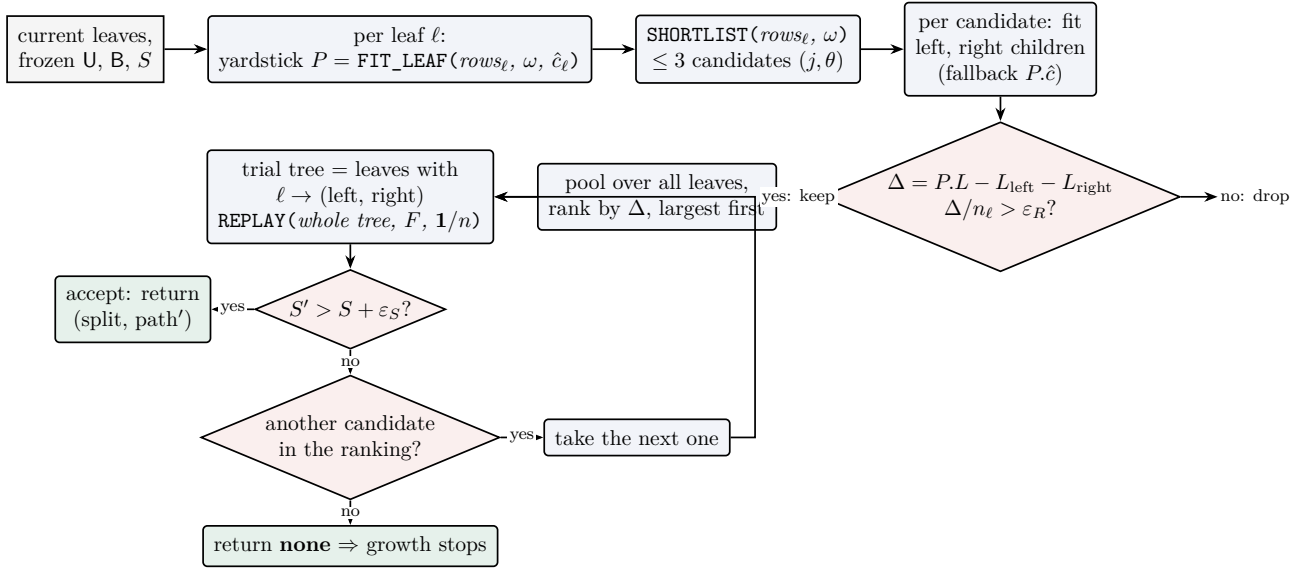
13 in the tree, leaf ℓ becomes the split node " $x_j \leq \theta$ " whose leaves are the two children;

 // the children keep their scores and rows from Algorithm 5

14 $\text{path} \leftarrow \text{path}'$; $\text{MEASURE}(\text{path})$; // the accepted tree's own path: new holdings, new S for the next round

15 **return tree**;

5 BEST_SPLIT: one round



Algorithm 6: BEST_SPLIT(leaves, S) (_best_split)

```

1 cand ← [];
2 foreach leaf ℓ = (node, rows, ω, depth) do
3   if depth ≥ D or ∑ ω < 2m*, m* = max(m, ⌈φ ∑ ω⌉) then
4     skip
5   P ← FIT_LEAF(rows, ω, node.ĉ) ; // same rows, current U: the yardstick; never
   written into the tree
6   foreach (j, θ) in SHORTLIST(rows, ω) do
7     left ← {t ∈ rows : xtj ≤ θ}; right ← rows \ left;
8     Lc ← FIT_LEAF(left, ω, P.ĉ); Rc ← FIT_LEAF(right, ω, P.ĉ);
9     Δ ← P.L - Lc.L - Rc.L;
10    if Δ/∑ ω > εR then
11      cand ← cand + (Δ, ℓ, j, θ, Lc, Rc) ; // regret gate: local, U frozen
12 sort cand by Δ, largest first ; // SPOT's criterion, over all leaves
13 foreach (Δ, ℓ, j, θ, Lc, Rc) in cand do
14   trial ← leaves with ℓ replaced by (Lc, left rows), (Rc, right rows) ; // other leaves
   unchanged
15   path' ← REPLAY(SCORES(trial, F), F, 1/n) ; // whole tree, whole fitting period,
   fees included
16   if path'.S > S + εS then
17     return (split, path') ; // Sharpe gate: in-sample; one acceptance per round
18 return none;

```

With plain splits the memberships are 0/1: left rows get ω , right rows get ω . With soft splits ($\kappa > 0$) a row goes left with weight $\mu(x) = \sum_{k: \theta_k \geq x} \pi_k$ and right with $1 - \mu(x)$, and a row can be in both children.

6 SHORTLIST and SCREEN: the candidates of one leaf

Algorithm 7: SHORTLIST($rows, \omega$) (`_shortlist`), v1 settings; the soft variant in the last lines

```

1  $m_* \leftarrow \max(m, \lceil \varphi \sum \omega \rceil)$ ; screened  $\leftarrow []$ ;
2 foreach feature  $j$  do
3    $\Theta \leftarrow$  midpoints of consecutive distinct values of  $x_{.j}$  on rows with mass  $\geq m_*$  on each side ;
   // plain candidates  $\theta_{(1)} < \dots < \theta_{(K)}$ 
4   if deciles then
5      $\Theta \leftarrow \Theta[\text{round}(q(K-1))]$  for  $q = 0.1, \dots, 0.9$  ; // quantiles by rank among the
     admissible thresholds
6    $Q, \text{per\_row} \leftarrow \text{SCREEN}(rows, \omega, x_{.j}, \Theta)$ ;
7   if refinement then
8      $\theta_o \leftarrow \arg \min_{\Theta} Q$ ;  $W \leftarrow$  plain thresholds strictly between the two elements of  $\Theta$ 
     neighbouring  $\theta_o$ ;
9      $Q_W, \text{per\_row}_W \leftarrow \text{SCREEN}(rows, \omega, x_{.j}, W)$ ;  $\theta_{\bullet} \leftarrow \arg \min_W Q_W$ ;
10     $se \leftarrow \sqrt{|rows|} \cdot \text{std}_t(\text{per\_row}_W[\theta_{\bullet}] - \text{per\_row}[\theta_o])$ ;
11    if  $Q(\theta_o) - Q_W(\theta_{\bullet}) > k \cdot se$  then
12      replace  $\theta_o$  by  $\theta_{\bullet}$  in  $\Theta$  (and its  $Q$ )
13  if  $\kappa = 0$  (plain) then
14    screened  $\leftarrow$  screened  $+\{(Q(\theta), j, \theta) : \theta \in \Theta\}$ 
15  else
16     $z_k \leftarrow (Q(\theta_k) - \min_l Q(\theta_l))/se_k$ ;  $\pi_k \propto e^{-z_k/\kappa}$  (or  $e^{-z_k^2/2\kappa^2}$ ); screened  $\leftarrow$  screened
     $+\{(\min Q, j, \Theta, \pi)\}$  ; // one soft split per feature
17 return the 3 entries of screened with the smallest  $Q$ ;
18 function SCREEN( $rows, \omega, values, \Theta$ ):
19   foreach  $\theta \in \Theta$  do
20     foreach side in ( $values \leq \theta, values > \theta$ ) do
21        $c_{\text{side}} \leftarrow$  clip(weighted mean of  $r_t$  on the side,  $\pm b$ ); regret $_t \leftarrow \omega_t \text{REG}(c_{\text{side}}, t)$  for  $t$  on
       the side;
22      $Q(\theta) \leftarrow \sum_t \text{regret}_t$ ; per_row $[\theta] \leftarrow (\text{regret}_t)_t$ ;
23   return  $Q, \text{per\_row}$ ;

```

7 RESCORE: the leaf scores again, on a given set of rows

Algorithm 8: RESCORE($tree, rows$) (`_refit_leaves`)

```

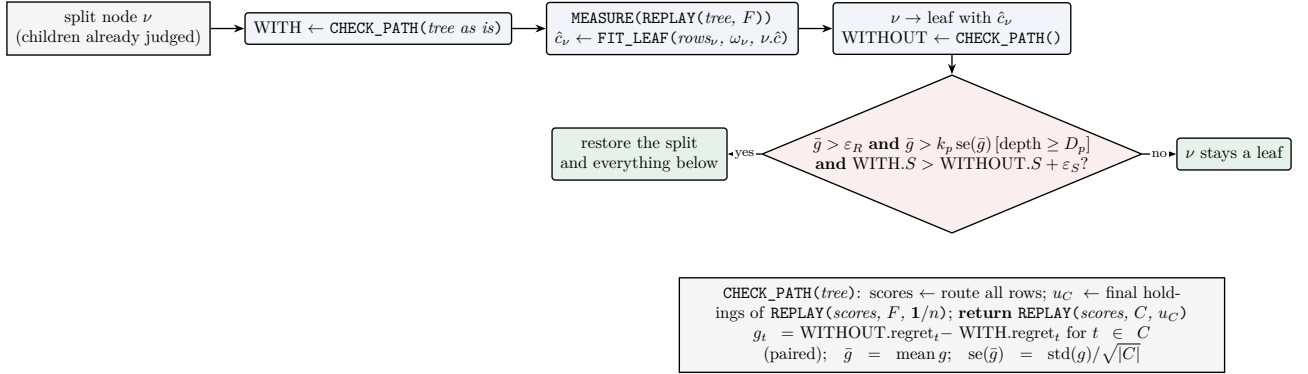
1 send the rows down the tree:  $rows_{\ell}$  for every leaf  $\ell$ ;
2 path  $\leftarrow \text{REPLAY}(\text{SCORES}(tree, rows), rows, 1/n)$ ; MEASURE(path) ; // holdings of the
   tree on these rows
3 foreach leaf  $\ell$  do
4    $\hat{c}'_{\ell} \leftarrow \text{FIT\_LEAF}(rows_{\ell}, \omega_{\ell}, \hat{c}_{\ell})$ 
5 path'  $\leftarrow \text{REPLAY}(\text{scores with the } \hat{c}', rows, 1/n)$ ;
6 if  $rows = F$  and path'. $S < \text{path}.S$  then
7   keep the old scores  $\hat{c}_{\ell}$ 
8 else
9   write every  $\hat{c}'_{\ell}$  into its leaf

```

Called twice. After growth, on F : the new scores replace scores fitted on the same rows, so the Sharpe

test applies. After pruning, on $F \cup C$: the old scores never saw C , so the new ones are always written.

8 PRUNE: bottom-up on the checking rows



Algorithm 9: PRUNE(root) (_prune)

```

1 function VISIT( $\nu$ ,  $\text{rows}_\nu$ ,  $\omega_\nu$ ,  $\text{depth}$ ):
2   if  $\nu$  is a leaf then
3     return
4   VISIT(left child, its rows, its weights,  $\text{depth}+1$ ); VISIT(right child, ...) ; // children first
5   WITH ← CHECK_PATH(tree as it is);
6   MEASURE(REPLAY(SCORES(tree, F), F, 1/n)) ; // holdings for the leaf fit
7    $\hat{c}_\nu \leftarrow \text{FIT\_LEAF}(\text{rows}_\nu, \omega_\nu, \nu.\hat{c})$ ; make  $\nu$  a leaf with  $\hat{c}_\nu$ , detaching the subtree;
8   WITHOUT ← CHECK_PATH(tree with  $\nu$  as a leaf);
9    $g_t \leftarrow \text{WITHOUT.regret}_t - \text{WITH.regret}_t$  for  $t \in C$ ;  $\bar{g} \leftarrow \text{mean}(g)$ ;  $\text{se} \leftarrow \text{std}(g)/\sqrt{|C|}$ ;
10   $\text{keep} \leftarrow \bar{g} > \varepsilon_R$  and  $\bar{g} > (k_p \text{se if depth} \geq D_p \text{ else } 0)$  and WITH.S > WITHOUT.S +  $\varepsilon_S$ ;
11  if keep then
12    restore the split and its subtree
13  else
14     $\nu$  stays a leaf with  $\hat{c}_\nu$ 
15 VISIT(root, F, 1, 0);
16 function CHECK_PATH(tree):
17   scores ← SCORES(tree, F  $\cup$  C);
18    $u_C \leftarrow$  final holdings of REPLAY(scores_F, F, 1/n); return REPLAY(scores_C, C,  $u_C$ ) ;
    // fees included; regrets and S on C only

```

With $k_p = 2$, $D_p = 1$: the root split is judged by the plain rule ($\bar{g} > \varepsilon_R$ and the Sharpe test); every deeper split must beat two standard errors. A split no checking row reaches has $\bar{g} = 0$ and is removed.

9 Where each quantity lives, and the two facts to remember

per date t	$U[t]$, $B[t]$, w_t , net_t , regret_t	written by REPLAY/MEASURE; read by REG; never stored in a node
per leaf ℓ	$\hat{c}_\ell$, n_ℓ , L_ℓ , rows_ℓ , ω_ℓ	rows and ω are training state; the frozen tree keeps $\hat{c}_\ell$ and n_ℓ
per split node	j , θ (soft: θ_k , π_k , spread), children	no score in use, no holdings
per round	S , the ranking of candidates	recomputed from scratch each round

- **Fees enter only through REPLAY:** $\text{fee}_t = f^\top |w_t - u|$ on the path. The regret sees them through $U[t]$ (frozen for a round) and through $|w - U[t]|$ inside the utility.
- **Holdings do not move while a candidate split is judged:** the regret gate compares one score against two scores on the same rows from the same U . The candidate's true path appears only in the Sharpe gate's replay, and becomes the new U through **MEASURE** when the split is accepted.

Code map

block	function in code/spo_tree.py
MAIN	<code>SPOPortfolioTree.fit</code>
REPLAY, fees, drift	<code>replay_path, _trade; SPOPortfolioTree._replay, _measure_from</code>
REG, FIT_LEAF	<code>_regret_rows, _score_prediction, _fit_leaf</code>
GROW, BEST_SPLIT	<code>_grow, _best_split, _scores, _leaves</code>
SHORTLIST, SCREEN	<code>_shortlist, _screen, _thresholds, _standard_error, _membership_of</code>
RESCORE	<code>_refit_leaves</code>
PRUNE, CHECK_PATH	<code>_prune (inner visit, checking_path)</code>
optimizer	<code>PortfolioOptimizer.solve (5^n enumeration), PortfolioOptimizer.utility</code>